

0. LLM API 协议格式

request Header:

```
~~~json { "model": "mymodelname", "message": [ { "role": "system", "content": "" }, { "role": "user", "content": "" }, {
"role": "assistant", "content": null, "toolcalls": [ { "id": "toolcallid001", "type": "function", "function": {
"name": "getweather", "arguments": { "city": "beijing" } } } ], { "role": "tool", "toolcallid": "toolcallid001", "content": "今天天
气晴朗", } ], "stream": true, "tools": [ { "type": "function", "function": { "name": "getweathae", "description": "用来查询用
户指定的天气", "parameters": { "type": "object", "properties": { "city": { "type": "string", "description": "用户要查询的天气
名称", }, "data": { "type": "string", "description": "用户要查询的具体日期", }, }, "required": { "city", "data", }, } } ],
"toolchoices": "auto", } ~~~
```

~~~markdown 一次大模型请求 Request | |—— model | |—— 类型: string | |—— 示例: "your-model-name"  
| |—— 含义: 指定本次调用使用哪个模型 | |—— messages | |—— 类型: list | |—— 含义: 发送给模型的完
完整对话上下文 | | | |—— [0] system 消息 | | |—— role = "system" | | |—— content = 系统提示词 | |
|—— 含义: 规定模型身份、行为规则、工具使用原则 | | | |—— [1] user 消息 | | |—— role = "user" | |
|—— content = 用户的问题 | | |—— 含义: 用户本轮输入 | | | |—— [2] assistant 消息 (模型要求调用工具
时) | | |—— role = "assistant" | | |—— content = null 或少量文本 | | |—— toolcalls / / / |—— 类
型: list / / / |—— 含义: 模型生成的一个或多个工具调用计划 / / / / / |—— [0] 一个工具调用 / / /
|—— id / / / / |—— 示例: "call001" | | | |—— 含义: 这一次工具调用的唯一标识 | | | | | | |
|—— type = "function" | | | |—— 含义: 调用的是函数型工具 | | | | | | |—— function | | |
|—— name | | | |—— 含义: 要调用的工具名称 | | | | | | |—— arguments | | | |—— 类
型: JSON 字符串 | | | |—— 示例: "{ \"city\": \"北京\" }" | | | |—— 含义: 模型为工具生成的实际参数 | |
| |—— 含义: 这不是最终回答, 而是模型要求 Runtime 执行工具 | | | |—— [3] tool 消息 (Runtime 执行
工具后追加) | |—— role = "tool" | |—— toolcallid = "call001" / / |—— 含义: 关联上一条
assistant.toolcalls 中的 id | |—— content | |—— 类型: 通常是字符串 | |—— 示例: "{ \"weather\": \"晴
\\\", \"temperature\": 28 }" | |—— 含义: 工具真实执行后返回的结果 | |—— 含义: 把工具结果交回模型继续生成
| |—— tools | |—— 类型: list | |—— 含义: 本次请求允许模型使用的工具说明 | | | |—— [0] 一个工具定
义 | | | |—— type = "function" | | |—— 含义: 这是函数型工具 | | | |—— function | | | |——
name | | |—— 示例: "getweather" / / |—— 含义: 工具名称 / / / |—— description / / |—— 含义:
说明工具做什么, 帮助模型判断是否调用 / / / |—— parameters | |—— 含义: 工具参数的 JSON Schema / /
| |—— type = "object" / / |—— 含义: 所有参数组成一个 JSON 对象 / / / |—— properties / / |——
含义: 定义允许传入的每一个参数 / / / / / |—— city / / |—— type = "string" / / / |—— 含义: city 必
须是字符串 / / |—— description / / / |—— 含义: 解释 city 参数用途 / / |—— minLength / / /
|—— 含义: 字符串最短长度 / / |—— maxLength / / |—— 含义: 字符串最长长度 / / / |—— required /
/ |—— 示例: [ "city" ] / / |—— 含义: 列出必须提供的参数 / / / |—— additionalProperties = false /
|—— 含义: 禁止生成 properties 中未定义的额外参数 / |—— toolchoice | |—— 示例: "auto" | |—— 含义:
控制模型如何选择工具 | auto: 模型自行判断 | none: 禁止调用工具 | 指定工具: 强制调用某个工具 | |——
temperature | |—— 示例: 0.2 | |—— 含义: 控制输出随机性 | |—— stream | |—— 示例: true | |——
含义: 是否使用流式响应 | |—— streamoptions |—— includeusage = true |—— 含义: 在流结束前额外返回
token 使用量 ~~~

# 1. RunContext — 拼接一次 run 的上下文

~~~树形 *runinput: RunInput* |—— *sessionid: str* | |—— 示例: "session-abc123" | |—— 含义: 本轮 run 归属的 session, 所有后续查询都基于它 |—— *userinput: str* | |—— 示例: "帮我查一下项目结构" | |—— 含义: 用户本轮输入的原始文本 |—— *agentname: Optional[str]* | |—— 示例: "code-reviewer" | |—— 含义: 显式指定本轮使用的 agent (不传则自动推断) |—— *skill\_name: Optional[str]* |—— 示例: "find-skills" |—— 含义: 显式加载某个 skill 的完整内容到 system prompt

RunContextFactory.assemble(run_input) 的组装过程:

|—— 1. *loadrecord(sessionid)* | |—— 查 *SessionRecord* 表, 拿到以下 session 级配置: | |—— *modelproviderid* → 模型供应商 ID | |—— *modelid* → 模型名称 | |—— *workspacepath* → 工作区物理路径 | |—— *sessiontype* → "coding" | "assistant" | |—— *permissionprofile* → "conservative" | "moderate" | "permissive" | |—— *contexttokens* → 上次调用的 input tokens (压缩预算参考) | |—— *thinkingenabled* → 是否开启深度思考 | |—— 2. *createadapter(sessionid, record)* → *ChatCompletionsAdapter* | |—— 查 *ProviderConfig* 表 → *apikey, baseurl* | |—— 查 *ModelSetting* 表 → *contextlength* | |—— *buildthinkingpayload(modelsetting)* → *extrapayload* | |—— 示例: {"thinking": {"type": "enabled", "effort": "medium"}} | |—— 含义: 思考模式的 provider 特定参数, 合并进每轮 LLM 请求 | |—— 3. *state = SessionStore.get(sessionid) or RunState()* | |—— 拿到最新的对话状态快照: | |—— *messages: list[ChatMessage]* → 历史对话记录 | |—— *step: int* → 当前步数计数 | |—— 自动压缩 (如果 messages 非空): | |—— *HistoryCompactor(adapter)* → 用 adapter 生成摘要 | |—— *CompactService.autocompactinmemory()* → 按 *contextlength* 预算裁剪 | |—— *contexttokens* → 上次消耗的 token 数 (判断触发阈值) | |—— *contextlength* → 模型最大上下文长度 | |—— *keeprecentcount=2* → 保留最近 2 轮对话 | |—— 4. *effectiveagentname = resolveeffectiveagentname(runinput, sessiontype)* | |—— *sessiontype == "coding"* → "softwareengineer" | |—— 含义: coding 模式默认用软件工程师 agent | |—— 否则 → *runinput.agentname* or "default" | |—— 含义: assistant 模式可用前端指定的 agent, 不传则用 default | |—— 5. *AgentDefinitionService.loaddefinition(effectiveagentname)* | |—— 从 agent 定义表加载 *AgentDefinition*: | |—— *systemprompt* → 基础系统提示词 | |—— *toolnames* → 允许使用的工具白名单 | |—— *description* → agent 描述 | |—— 6. *ContextAssembler.assemble(...)* → 合并运行时系统提示词 | |—— *loadworkspacecontext(workspacepath, sessiontype)* | |—— 工作区根目录 / AGENTS.md → *localrulestext* (项目级指令) | |—— 仅 assistant 模式: | |—— 工作区根目录 / SOUL.md → *agentsoultext* (人设描述) | |—— 工作区根目录 / USER.md → *userprofiletext* (用户画像) | |—— *SkillContextService.buildruntimedefinitionwithskills()* | |—— *buildskillcatalogprompt()* | |—— 含义: 始终往 *system prompt* 末尾追加所有可用 skill 的概要列表 | |—— 如果 *runinput.skillname* 非空: | |—— *loadskillcontent(skillname)* → 加载该 skill 的完整内容 | |—— 含义: 把 skill 的详细指令注入 system prompt | → 返回合并后的 *systemprompt* 文本 | |—— 7. *runtimedefinition = definition.modelcopy(update={"systemprompt": newprompt})* | |—— 含义: 复制基础 *AgentDefinition*, 只替换 *systemprompt* 为合并后的版本 | |—— *toolnames* 保持不变 (来自 *definition* 的白名单) | |—— 上游通过 *agentprofile.toolnames* 读取 | |—— 8. *approvalpolicy = resolveapprovalpolicy(record)* | |—— *permissionprofile* → *PROFILES* 查表 → *ApprovalPolicy* | |—— *conservative* → *ApprovalPolicy.ALWAYS* | |—— *moderate* → *ApprovalPolicy.HIGHRISKONLY* | |—— *permissive* → *ApprovalPolicy.NEVER* | |—— 9. 最终返回 *RunContext* | |—— *state: RunState* | |—— 示例: *messages=[ChatMessage(role="system", content="你是一个助手"), ...]*, *step=3* | |—— 含义: 本轮启动时的对话状态快照 (已压缩) | |—— *agentprofile: AgentDefinition* | |—— 示例: *systemprompt="你是软件工程师...\n可用技能: ..."*, *toolnames=["readfile", "writefile"]* | |—— 含义: 最终生效的 agent 定义 (*system prompt* + 工具白名单) | |—— *adapter: ChatCompletionsAdapter* | |—— 示例:

ChatCompletionsAdapter(*baseurl*="https://api.example.com", *model*="gpt-4") | —— 含义: 已绑定 session 模型配置的通讯器, 下游直接调 *adapter.generate()* | —— *approvalpolicy: ApprovalPolicy* | —— 示例: *ApprovalPolicy.HIGHRISKONLY* | —— 含义: 工具审批策略, 传到 *toolrunner* 控制审批行为 | —— *effectiveagentname: str* | —— 示例: "softwareengineer" | —— 含义: 本轮实际使用的 agent 名称 (落库 + 前端显示用) | —— *workspacepath: str* | —— 示例: "/Users/wangxu/projects/my-app" | —— 含义: 物理工作区路径 (文件工具 + VFS 的根目录) | —— *session_type: str* | —— 示例: "coding" | —— 含义: 会话类型 (决定 agent 选择 + prompt 组装 + 沙箱策略) ~~~

2. ToolTracer — 工具执行观察者

~~~树形 *ToolTracer*(*db*, *runstore*, *approvalstore*, *sessionid*, *runid*, *run\_input*)

构造参数: | —— *db: Session* | —— 含义: 统一数据库会话, 每次回调内部 *commit* | —— *runstore: RunTraceStore* | —— 含义: *run trace* 持久化存储, 提供以下方法: | —— *createtoolcall(runid, toolname, toolcallid, inputjson) → recordid* | —— 含义: 工具开始执行时创建 "running" 状态记录 | —— *finishtoolcall(recordid, status, resultjson)* | —— 含义: 工具结束时更新状态为 "completed" / "failed" / "timeout" | —— *saveruntrace(sessionid, runid, agentname, userinput, reply, events)* | —— 含义: 保存完整 run 记录 + 所有事件到 *SessionRunEventRecord* 表 | —— *savepartialrun(...)* | —— 含义: 保存未完成的 run 记录 (中断/审批暂停场景) | —— *approvalstore: SqliteApprovalStore* | —— 含义: 审批记录持久化存储 | —— *sessionid: str* | —— 含义: 所属会话 ID | —— *runid: str* | —— 含义: 本轮 run 的唯一 ID | —— *run\_input: RunInput* | —— 含义: 本轮用户输入, 透传备用

三个回调方法 (挂载到 *asynchandletool\_calls*):

| —— *ontoolstart(toolname, toolcallid, inputjson) → recordid* | —— 触发时机: 每个工具开始执行前 | —— 副作用: *runstore.createtoolcall()* + *db.commit()* | —— *ontoolfinish(recordid, status, resultjson)* | —— 触发时机: 每个工具执行结束后 | —— *status* 取值: "completed" / "failed" / "timeout" | —— 副作用: *runstore.finishtoolcall()* + *db.commit()* | —— *onapprovalrequired(toolcallid, toolname, arguments, savedmessages, eventindex, batchid) → approvalid* | —— 触发时机: 工具需要人工审批时 | —— 副作用: *approval\_store.create()* + *db.commit()* ~~~

## 3. AgentRunner — 智能体发动机构造

~~~树形 *AgentRunner*( *state*=*ctx.state*, *agentprofile*=*ctx.agentprofile*, *modeladapter*=*ctx.adapter*, *toolregistry*=*buildrunregistry(childdispatcher, statuschecker, childwaiter)*, *approvalpolicy*=*ctx.approvalpolicy*, *sessiontype*=*ctx.session\_type*, )

构造参数: | —— *state: RunState* | —— 含义: 当前对话状态快照 (messages + step), 来自 *ctx.state* (已压缩) | —— *agentprofile: AgentDefinition* | —— 含义: 最终生效的 agent 定义, 包含合并后的 *systemprompt* + *toolnames* | —— *modeladapter: ChatCompletionsAdapter* | —— 含义: 大模型通讯器, *AgentRunner* 直接调 *adapter.asyncstreamgenerate()* | —— *toolregistry: ToolRegistry* | —— *buildrunregistry()* 构建, 注入三个闭包回调: | —— *childdispatcher: ChildRunLauncher.createlauncher(runid, sessionid)* | —— 含义: 子 Agent 启动

回调 (供 *agentbridge* 工具使用) | |—— *statuschecker: ChildRunLauncher.createstatuschecker()* | |—— 含义: 子 Agent 状态查询回调 | |—— *childwaiter: ChildRunLauncher.createwaiter()* | |—— 含义: 子 Agent 等待完成回调 |—— *approvalpolicy: ApprovalPolicy* | |—— 含义: 工具审批策略, 传到 *asynchandletoolcalls* 控制审批逻辑 |—— *sessiontype: str* |—— 含义: 会话类型, 传到 *asynchandletool_calls* 决定安全中间件管道 |—— "coding" → *MiddlewarePipeline([SandboxMiddleware()])* → 启用沙箱 |—— 其他 → *MiddlewarePipeline([])* → 空管道 ~~~

4. RunSSEBridge — SSE 协议适配

~~~树形 *RunSSEBridge*( *ctx*, *observer*, *agentrunner*, *runid*, *run\_input*, *persist* )

构造参数: |—— *ctx: RunContext* | |—— 含义: 第 1 步组装的完整运行背景 (*effectiveagentname* 用于 start 帧) |—— *observer: ToolTracer* | |—— 含义: 第 2 步构建的工具追踪器, 三个回调传给 *RunLifecycle* |—— *agentrunner: AgentRunner* | |—— 含义: 第 3 步构建的执行发动机 |—— *runid: str* | |—— 含义: 本轮 run 的唯一 ID (start 帧 / paused 帧用到) |—— *runinput: RunInput* | |—— 含义: 用户输入 (*sessionid* + *agent\_name* 用于 start 帧) |—— *persist: RunRecorder* |—— 含义: 终态持久化入口

---

*stream()* 方法执行流程:

*yield* start 帧 | |—— *sessionid, runid, agentname* | |—— 含义: 告诉前端"一轮新 run 开始了", 准备接收后续帧 | |—— 构建 *RunLifecycle(params)* | |—— *ctx* → *ctx* 完整传入 | |—— *agentrunner* → 发动机 | |—— *persist* → 持久化 | |—— *runinput* → 用户输入 | |—— *runid* → run ID | |—— *ontoolstart* ← *observer.ontoolstart* | |—— *ontoolfinish* ← *observer.ontoolfinish* | |—— *onapprovalrequired* ← *observer.onapprovalrequired* | |—— *skipusemessage* → 默认 False (resume 时为 True) | |—— *eventindex* → 默认 0 (resume 时从上次断点接着编号) | |—— *initialevents* → 默认空 (resume 时补 *toolresult* 事件) | |—— *appendevents* → 默认 False (resume 时为 True, 往已有 run 追加事件) | |—— *updatesessionsnapshot* → 默认 True (子 Agent 设为 False, 防止污染主 session) | |—— *async for item in lifecycle.iterate():* | |—— *TextDeltaItem* → SSE "delta" 帧 | |—— *type="delta"* | |—— *data={content: "文本片段"}* | |—— *ThinkingDeltaItem* → SSE "thinkingdelta" 帧 | |—— *type="thinkingdelta"* | |—— *data={content: "思考片段"}* | |—— *RunEventItem* → SSE "runevent" 帧 | |—— *data=event.modeldump()* | |—— *index, type, content, toolname, toolcallid* | |—— 可能的 *type: assistanttoolcall / toolresult / toolerror / thinking / finalanswer / approvalrequired / toolprogress* | |—— *RunStatusItem* → 终态帧 |—— *status == PAUSED* → SSE "paused" 帧 | |—— *data={runid}* → 前端跳转审批界面 |—— *status 其他* → SSE "end" 帧 |—— *data={state: agentrunner.state.model\_dump()}* |—— *messages + step* |—— 含义: 前端展示最终状态摘要 ~~~

---

## 5. RunLifecycle.iterate() — 执行生命周期

---

~~~树形 *iterate()* 内部准备

|—— *replytext = ""* → 累积最终回复文本 |—— *thinkingbuf = ""* → 累积思考文本片段 |—— *events: list[RunEvent]* → 正式事件账本 | |—— 初始值 = *initialevents* (resume 场景预先填入, 默认空) | |—— *flushthinking()* 内联异步生成器 |—— 作用: 把 *thinkingbuf* 收束成一条 *RunEvent(type="thinking")* |—— 创建 *RunEvent(index=len(events),*

type="thinking", content=thinkingbuf) |—— events.append(event) |—— thinkingbuf 清空 |—— yield
RunEventItem(event=event)

主循环: async for item in agentrunner.asyncstream_run(...)

传入参数: |—— runinput → 用户输入 |—— ontoolstart → ToolTracer.ontoolstart |—— ontoolfinish → ToolTracer.ontoolfinish |—— onapprovalrequired → ToolTracer.onapprovalrequired |—— skipusermessage → 是否跳过自动追加 user message |—— eventindex → 事件序号起点 |—— runid → run ID |—— workspacepath → 工作区物理路径

对每个产出的 item 分类处理:

|—— isinstance(item, str) | |—— replytext += item → 累积到最终回复 | |—— yield
TextDeltaItem(content=item) → 实时透传文字片段 | |—— isinstance(item, StreamChunk) and type ==
"thinkingdelta" | |—— thinkingbuf += item.thinkingdelta → 攒着, 不收束 | |—— yield
ThinkingDeltaItem(content=chunk) → 实时透传思考片段 | |—— isinstance(item, RunEvent) |—— type ==
"toolprogress" | |—— yield RunEventItem(event=item) → 直接透传, 不进 events 列表 | |—— type 其他
(assistanttoolcall / toolresult / toolerror / finalanswer / approval_required) |—— flushthinking() → 先把积攒的
thinking 收束成事件 |—— events.append(item) → 记入正式事件账本 |—— yield RunEventItem(event=item) → 透传

循环结束后的终态收口

正常结束 (循环自然退出): |—— flushthinking() → 最后一次收束 thinking |—— 判断终态: | |—— events 中有
approvalrequired → PAUSED | |—— 否则 → COMPLETED |—— persist.finalizerun(status, events, replytext,
state, usage) | |—— 写 SessionRunRecord (run 摘要) | |—— 写 SessionRunEventRecord (每条 event 一行)
| |—— 更新 SessionRecord.statejson (最新对话状态) | |—— completed → VFS.commitall() → staged 文件落
盘 | |—— 更新 runstatus = "completed" |—— yield RunStatusItem(status=status) → RunSSEBridge 据此发 end
/ paused 帧

被取消 (GeneratorExit / asyncio.CancelledError): |—— flushthinking() |——
persist.finalize_run(status=CANCELLED) | |—— CANCELLED / FAILED → VFS.discard() → staged 文件删除 |
|—— 写部分事件到 DB |—— raise → 往上抛, 让 Starlette 处理清理

异常 (Exception): |—— flushthinking() |—— persist.finalize_run(status=FAILED) | |—— VFS.discard() |——
raise → 往上抛 ~~~

6. AgentRunner.asyncstreamrun() — 引擎内部

~~~树形 while True 循环体 (一次迭代 = 一轮 LLM 对话):

1. 组装请求 request = buildmodelrequest(agentprofile, state, toolregistry) |—— system\_prompt +  
state.messages + tools

2. 消费 LLM 流 (逐帧处理 StreamChunk) `async for chunk in adapter.asyncstreamgenerate(request):` | | ——  
`type="done" + usage` | | —— `self.lastusage = chunk.usage` → 只记录, 不 `yield` | | ——  
`type="thinkingdelta"` | | —— `yield chunk` → 原样透传给 `RunLifecycle` | | —— `type="contentdelta"` /  
| —— `yield chunk.contentdelta` → 拆成裸字符串 `yield` | | —— `rawreplychunks.append(...)` → 攒着, 备用 |  
| —— `type="toolcall/delta"` | —— 按 `index` 分组收进 `toolcall/buffers` | —— `buf["id"] ← tc.id` | ——  
`buf["namechunks"] ← function.name` 片段 | —— `buf["argschunks"] ← function.arguments` 片段
3. 判定 `finishreason` (LLM 流结束后) | —— `finishreason == "toolcalls"` | | —— 拼接 `ToolCall` 列表 | |  
| —— `id = buf["id"]` | | | —— `function.name = "".join(buf["namechunks"])` | | | ——  
`function.arguments = "".join(buf["argschunks"])` | | | —— 注意: `name` 和 `args` 可能跨多帧, 靠 `index` 归组  
| | | | —— 构造 `assistant ChatMessage` | | | —— `content = rawreplychunks` 拼接 (可能为 `None`) |  
| | | | —— 含义: 模型可能在 `toolcalls` 前先说一段话 (`content lead-in`) | | | —— `toolcalls =` 拼接好的列  
表 | | | | —— 执行工具 `asynchandletoolcalls(...)` | | | —— `yield RunEvent` → 实时透传工具事件 | |  
| —— `pausedforapproval` → `break` (审批中断) | | | —— 否则 → 更新 `eventindex` → `continue` 下一轮 |  
| | | —— 工具结果追入 `state.messages` → `continue` | | —— 其他 (`stop / length / None`) | ——  
`buildreply(rawreply, eventindex)` | | —— `reply = rawreply.strip()` | | —— `finalevent =`  
`RunEvent(type="finalanswer")` | | —— `assistantmessage = ChatMessage(role="assistant")` | | —— `yield`  
`final_event` → `break`

关键细节:

| —— `rawreplychunks` 和 `toolcall/buffers` 每轮重置 | | —— 含义: 每轮 LLM 调用独立, 不跨轮累积 | | ——  
`finishreason` 只取最后一个非 `None` 值 | | —— 含义: 流中间帧 `finishreason` 为 `None`, 最后一帧才有值 | | ——  
`content lead-in bug` 修复 | | —— 模型吐了 `content` 再吐 `toolcalls` → `rawreplychunks` 非空 | → `assistant message`  
同时保留了 `content` 和 `toolcalls` | | —— `yield` 出去的三种形态 | —— `str` → 正文片段 (`RunLifecycle` 攒起来) | ——  
`StreamChunk` → `thinking_delta` 片段 (`RunLifecycle` 攒起来) | —— `RunEvent` → 正式事件 (`RunLifecycle` 透传或收束  
`thinking` 后再传) ~~~

## 7. ChatCompletionsAdapter — 大模型通讯

~~~树形 一、初始化阶段 (`RunContextFactory` 构建, 整个 `run` 复用)

`ChatCompletionsAdapter( apikey, baseurl, model, extrapayload, thinkingstyle, )` | —— `baseurl` → 查
`SessionRecord.model/providerid` → `ProviderConfig.baseurl` | —— `apikey` → 查 `ProviderConfig.apikey` | —— `model`
→ `SessionRecord.modelid` (用户选定的模型名) | —— `extrapayload` → `buildthinkingpayload(thinkingstyle, enabled,`
`effort)` | | —— `deepseekstyle: {"thinking": {"type": "enabled"}, "reasoningeffort": "medium"}` | | ——
`sensenovastyle: {"reasoningeffort": "medium"}` | | —— `none: {}` | —— `thinkingstyle` → 查
`ModelSetting.thinking_style`

二、构造 HTTP 请求

每次 `asyncstreamgenerate(request: ModelRequest)` 被调用时:

`headers:` | —— `Accept: "text/event-stream"` → 告诉服务器返回 SSE 流 | —— `Authorization: "Bearer {api_key}"` →
身份认证 | —— `Content-Type: "application/json"` → 请求体格式

payload: |—— model: request.config.model or self.model |—— messages: [systemprompt, *state.messages]
 |—— stream: True → 强制流式 |—— tools: request.tools → 工具 JSON Schema 列表 |—— toolchoice: "auto"
 |—— temperature / topp / maxoutputtokens → 可选参数 |—— streamoptions: {"includeusage": True} → 要求返回
 token 用量 |—— provideroptions → 合并 request.config.provideroptions |—— extrapayload → 合并 thinking
 payload

三、建立连接（带重试）

max_retries = 3, delay = 1s → 2s → 4s（指数退避）

async with httpx.AsyncClient(timeout=2100) as client: ctx → httpx Stream 上下文管理器（控制连接开关） response
 → 响应头对象（提供 aiter_lines() 逐行读 body）

尝试 4 次：

```
ctx = client.stream("POST", url, headers=headers, json=payload) → 发请求
response = await ctx.__aenter__() → 等握手
response.raise_for_status() → 检查 200
成功 → break
失败 → await ctx.__aexit__() 清理 + await asyncio.sleep(delay) 重试
```

四、逐行解析 SSE

async for line in response.aiter_lines(): |—— 空行 / 非 "data:" 开头 → 跳过 |—— text == "[DONE]" → break |——
 JSON 解析 chunk | |—— choices 为空 (usage 帧) | |—— yield StreamChunk(type="done",
 usage=ModelUsage(...)) | |—— choices 非空 (内容帧) |—— choice = choices[0] |—— delta = choice["delta"]
 |—— finishreason = choice["finishreason"] |—— parsedelta(delta, finishreason) → 0~N 个 StreamChunk

五、parsedelta — 原始 delta → StreamChunk 打标签

delta 字段 → StreamChunk 产出

reasoningcontent →

type="thinkingdelta", thinkingdelta=值 或 thinkingcontent 或 reasoning

content → type="contentdelta", contentdelta=值, finishreason=null/"stop"/"toolcalls"

toolcalls → type="toolcalldelta", toolcalldelta=整个原始delta, finishreason=null/"stop"/"tool_calls"

为空 → type="done", finishreason="stop"/"toolcalls"（兜底：最后一帧 delta 为空时）

注意：一个 delta 可以同时有 content + tool_calls → 产出两个 StreamChunk

六、原始 choices → StreamChunk 对照表

原始 choices（模型返回的每一帧）： choices[0]: delta: content: "你好" → contentdelta reasoningcontent: "让我先想一想..." → thinkingdelta toolcalls: [{index, id, function}] → toolcalldelta finishreason: null / "stop" / "toolcalls"

StreamChunk 5 种产出:

type="contentdelta" contentdelta: "你好" finishreason: null (中间帧) 或 "stop"/"toolcalls" (最后一帧)

type="thinkingdelta" thinkingdelta: "让我先想一想..."

type="toolcalldelta" toolcalldelta: {"toolcalls": [...]} ← 存整个原始 delta finishreason: null 或 "tool_calls"

type="done" finishreason: "stop" 或 "toolcalls" ← 空 delta 兜底

type="done" usage: {inputtokens, outputtokens, total_tokens} ← usage 帧 ~~~

速查表

~~~树形 完整调用链

POST /run/stream → RunService.stream() | |—— ① RunContextFactory.assemble() | |—— 查 SessionRecord  
→ ProviderConfig + ModelSetting → 构建 adapter | |—— 读旧 state → HistoryCompactor.autocompact → 压缩  
后 state | |—— AgentDefinition → ContextAssembler (AGENTS.md + skill) → 合并 systemprompt | |——  
RunContext{state, agentprofile, adapter, policy, workspace, sessiontype} | |—— ② ToolTracer(db, runstore,  
approvalstore, sessionid, runid) | |—— ontoolstart / ontoolfinish / onapprovalrequired | |—— ③  
AgentRunner(state, agentprofile, adapter, toolregistry, policy) | |—— ④ RunSSEBridge.stream() | |—— yield  
"start" {runid} | |—— RunLifecycle.iterate() | |—— 收到 str → replytext+=item → yield TextDeltaItem → SSE  
"delta" | |—— 收到 thinkingdelta → thinkingbuf+=item → yield ThinkingDeltaItem → SSE "thinkingdelta" /  
| |—— 收到 RunEvent | |—— toolprogress → yield (不入 events) | |—— toolresult / toolerror →  
replytext="" | |—— 其他 → flush thinking → events.append → yield | |—— 终态 → persist.finalizerun() → 落  
库 + VFS → yield "end"/"paused" | |—— ⑤ AgentRunner.asyncstreamrun() — while True 循环 | |——  
buildmodelrequest() → adapter.asyncstreamgenerate() | |—— thinkingdelta → yield 给 RunLifecycle | |  
| |—— contentdelta → yield 出去 + 攒进 rawreplychunks | |—— toolcalldelta → 按 index 攒进 toolcallbuffers  
| |—— done(usage) → 存 self.lastusage | |—— finishreason == "toolcalls" | |—— 拼 ToolCall → 构造  
assistant (content + toolcalls) | |—— yield assistanttext event (lead-in 非空时) | |——  
asynchandletoolcalls() → 审批预检 → 并发执行 → ToolBatchResult | |—— state.messages 追入 toolmessages  
→ paused? → break : continue | |—— continue 下一轮 | |—— finishreason == "stop" | |—— buildreply()  
→ finalanswer event | |—— state.messages.append(assistant) | |—— break | |—— ⑥  
ChatCompletionsAdapter | |—— 构造 HTTP: headers + payload (model, messages, stream:true, tools,  
extrapayload) | |—— httpx.stream("POST") + 重试 3 次 (1s→2s→4s) | |—— aiterlines() → choice[0].delta +  
finishreason → parsedelta() | |—— reasoningcontent / thinkingcontent / reasoning → thinkingdelta | |——  
content → contentdelta | |—— toolcalls → toolcalldelta (存整个原始 delta) | |—— 空 → done | |—— ⑦  
落库: RunRecorder.finalizerun() | |—— COMPLETED → saveruntrace(reply) + savestate(state) + VFS.commit | |——  
PAUSED → savepartialrun (不注入空消息) | |—— CANCELLED → savepartialrun + 补 assistant (流中断时) +  
VFS.discard | |—— FAILED → savepartialrun + 补 assistant (流中断时) + VFS.discard

RunEvent 8 种

|—— assistanttext 工具前文字 content = lead-in 文本 |—— assistanttoolcall 工具调用启动 content = 工具参数  
JSON |—— toolresult 工具执行成功 content = 结果文本 |—— toolerror 工具执行失败/超时 content = 错误消息



|—— *finalanswer* 模型最终回答 *content* = 答案文本 |—— *approvalrequired* 需要人工审批 *content* = *approvalid*  
 |—— *thinking* 思考收束块 *content* = 思考全文 |—— *tool\_progress* 工具进度片段 不进 *events* 列表

StreamChunk 4 种

|—— *thinkingdelta* 思考增量 *thinkingdelta* = 文本 |—— *contentdelta* 正文增量 *contentdelta* = 文本 + *finishreason*  
 |—— *toolcalldelta* 工具调用增量 *toolcalldelta* = 整个原始 *delta* |—— *done* 流结束 *finishreason* 或 *usage*

*state.messages* 累加

*user* → *assistant*(*content*=*lead-in*, *toolcalls*=[...]) → *tool* × *N* (每个 *tool* 绑定 *toolcall\_id*) → ... (多轮循环) →  
*assistant*(*content*=最终回答) → *break*

落库四态

|—— *COMPLETED* → *saveruntrace* + *savestate* + *VFS.commit* |—— *PAUSED* → *savepartialrun* (不注入空  
*assistant*) |—— *CANCELLED* → *savepartialrun* + 补 *assistant* (流中断时) + *VFS.discard* |—— *FAILED* →  
*savepartial\_run* + 补 *assistant* (流中断时) + *VFS.discard* ~~~